

CONFIDENTIAL INTERNAL MEMO: Next-Generation MLOps and Distributed System Architectures

1.0 Introduction to Pipeline V4 The migration to Pipeline V4 represents a fundamental shift in how we handle distributed machine learning workloads. Historically, our monolithic architecture resulted in high latency during inference and massive GPU idle times. The new architecture transitions our infrastructure entirely to Kubernetes-native orchestration, utilizing Terraform for stateful infrastructure provisioning across multi-cloud environments.

2.0 Real-Time Data Ingestion and Streaming The core bottleneck of V3 was batch processing. In V4, we have implemented a Real-Time Sales Analytics and Telemetry platform. Raw data is ingested via Apache Kafka clusters utilizing compacted topics for fault tolerance. This stream is processed in real-time by Spark Streaming clusters, which handle windowed aggregations and feature engineering before writing the finalized feature vectors into an Amazon S3 data lake. This architecture guarantees a maximum latency of 400 milliseconds from raw event generation to feature availability.

3.0 Model Training and Vectorization Strategy For generative AI workloads, including our proprietary Large Language Model (LLM) fine-tuning operations, we utilize distributed data parallelism. Checkpoints are automatically saved to AWS S3. Furthermore, our Retrieval-Augmented Generation (RAG) products rely heavily on advanced vector database management. Document embeddings are generated using the text-embedding-3-small architecture and stored in ChromaDB and FAISS instances. To maintain retrieval accuracy, we enforce a strict semantic chunking policy: documents are split into 1000-token chunks with a 200-token overlap, ensuring contextual integrity across paragraph boundaries.

4.0 Model Serving and Containerization Models are no longer served via legacy Flask wrappers. Production models, including our Fashion MNIST Deep Learning Classifier and our LLM routing engines, are packaged inside Docker containers and served via FastAPI. This allows for asynchronous request handling. Kubernetes handles horizontal pod autoscaling based on custom Prometheus metrics, specifically scaling up when GPU memory utilization exceeds 85%.

5.0 Observability and The Evaluation Triad Deployment is only half the lifecycle. MLflow is integrated universally across the V4 pipeline to track hyperparameters, model weights, and prompt inference latencies. To prevent LLM hallucination in production, all generative models are gated by an automated CI/CD pipeline. This pipeline executes an LLM-as-a-Judge evaluation, grading outputs on the RAG Triad: Context Relevance, Groundedness, and Answer Relevance. A groundedness score below 0.80 will automatically trigger a build failure, preventing the model from reaching the production cluster.

Once you ingest this PDF, try asking the system: *"What is the exact groundedness score threshold required to prevent a build failure in Pipeline V4?"* or *"Explain the latency improvements in the data ingestion layer."*